



ChatBot
Aula 09

Prof. Me Daniel Vieira



Atividade

1- Criar um chatbot para realizar a leitura de um pdf com informações sobre agricultura, tipos de solo, que auxilie o agricultor na escolha da semente e como preparar o solo para o plantio, bem como definir os limites de temperatura para o solo e irrigação de acordo com a semente plantada.

Carregar o pdf docs agricultura

Atividade

Você deve criar um banco de dados vetorial para armazenar as informações do pdf carregado conforme o exemplo da aula anterior

Perguntas a serem feitas para chatbot assistente Agro

- 1) Por que é importante preparar o solo adequadamente antes do plantio?**
- 2) Quais são os principais tipos de solo e como prepará-los para o cultivo?**
- 3) Quais sensores podem ser utilizados para monitorar a umidade do solo e como funcionam?**
- 4) Como é possível ajustar o pH do solo para atender às necessidades das diferentes culturas?**
- 5) Quais são os principais fertilizantes utilizados e qual a diferença entre fertilizantes orgânicos e químicos?**
- 6) Qual é o processo recomendado para a correção e preparo do solo antes do plantio?**
- 7) Como garantir a fertilidade do solo para diferentes tipos de culturas?**
- 8) Quais são os principais fatores que devem ser levados em consideração ao planejar a irrigação do solo?**
- 9) Quais são as melhores práticas para evitar a compactação do solo e a erosão?**
- 10) Onde um agricultor pode obter mais informações sobre preparo do solo e escolha de sementes?**

Atividade

Subir o código no Teams e criar um relatório documentando os passos para criar o chatbot

ChatBot com RAG



Retrieval - Augmented Generation

Tecnicamente RAG é uma estrutura de IA para recuperar fatos de uma base de conhecimento externa para fundamentar grandes modelos de linguagem (LLMs) nas informações mais precisas e atualizadas e para fornecer aos usuários uma visão sobre o processo gerador de LLMs.

Os LLMs enfrentam uma série de desafios atualmente, como os seguintes:

- Fornecer informações falsas quando não possuem uma resposta adequada.
- Oferecer informações desatualizadas ou genéricas quando os usuários solicitam respostas específicas e atualizadas.
- Gerar respostas baseadas em fontes não confiáveis.
- Criar respostas imprecisas devido à confusão terminológica, quando diferentes fontes de treinamento utilizam a mesma terminologia para descrever conceitos distintos.

Chroma DB

ChromaDB é um banco de dados vetorial de código aberto projetado para armazenar, indexar e recuperar vetores de alta dimensão de maneira eficiente. Ele é amplamente utilizado para tarefas como **busca semântica, sistemas de recomendação e aplicações de IA generativa**, onde embeddings de texto, imagem ou áudio são usados para recuperar informações relevantes.

Código

```
1 # Aula 07 - Chatbot utilizando o Gemini
2
3 # Instalação das bibliotecas
4 ! pip install -q --upgrade pip jedi
5 ! pip install -q llama-index llama-index-readers-file llama-index-llms-google-genai llama-index-embeddings-google-genai
```

```
1
2 # Rode esta célula antes de usar o Gemini com LlamaIndex
3 %pip install -q nest-asyncio
4 import nest_asyncio
5 nest_asyncio.apply()
6 import os
7 from google.colab import userdata
8 from llama_index.llms.google_genai import GoogleGenAI
9 from llama_index.embeddings.google_genai import GoogleGenAIEmbedding
10 from llama_index.core import Settings
11
12
13 # Pega a variável de ambiente
14 os.environ['Gemini_chatbot'] = userdata.get('Gemini_chatbot')
15 api= os.environ['Gemini_chatbot']
16 #
```

Código

Verificando informações do documento carregado

```
1 # Cria a variavel chamada llm
2 llm = GoogleGenAI(
3     model='gemini-2.5-flash-lite',
4     api_key=api
5 )
6
7 Settings.llm = llm
8 #Settings.embed_model = embed_model
```

1. Envie arquivo para a base de conhecimento utilizando a técnica RAG

Cria uma pasta chamada documentos no Colab e envie seus arquivos para lá

```
1 from google.colab import files
2 import os
3 os.makedirs("/contents/documentos",exist_ok=True)
4 print('Pasta criada em /content/documentos')
```

Código

[12]

✓ Os

```
1 from google.colab import files
2 import os
3 os.makedirs("/content/documentos", exist_ok=True)
4 print('Pasta criada em /content/documentos')
```

▼

Pasta criada em /content/documentos



[14]

✓ Os



```
1 #Leitura dos arquivos
2 from llama_index.core import SimpleDirectoryReader
3 documentos = SimpleDirectoryReader(input_dir='/content/documentos')
```

Código

[15]

✓ 1s

```
1 docs = documentos.load_data()  
2 print(f'Quantidade de documentos carregados {len(docs)}')
```

▼

Quantidade de documentos carregados 10

[16]

✓ 0s

```
1 # Exibindo os metadados do documento  
2 print(docs[0].get_metadata_str())
```

▼

```
page_label: 1  
file_name: serenatto_cafes_especiais.pdf  
file_path: /content/documentos/serenatto_cafes_especiais.pdf  
file_type: application/pdf  
file_size: 133957  
creation_date: 2026-04-01  
last_modified_date: 2026-04-01
```

Código

1 #Quebrando o documento em pedaços (nodes)

2 #importando a biblioteca

3 from llama_index.core.node_parser import SentenceSplitter

4 node_parser=SentenceSplitter(chunk_size=1200) # tamanho da divisao

5 # Fazer a divisao do documento carregado com base no chunk size

6 nodes = node_parser.get_nodes_from_documents(docs,show_progress = True)

7 print(f'Quantidade de nodes gerados: {len(nodes)}')

8

... Parsing nodes: 100%  10/10 [00:00<00:00, 232.12it/s]

Quantidade de nodes gerados: 10

Código



```
1 # Configurando o LLM Gemini e o modelo de embeddings
2 from llama_index.llms.google_genai import GoogleGenAI
3 from llama_index.embeddings.google_genai import GoogleGenAIEmbedding
4 from llama_index.core import Settings
5
6 llm = GoogleGenAI(
7     model = 'gemini-2.5-flash-lite',
8     api_key = api
9 )
10
11 embed_model = GoogleGenAIEmbedding(
12     model='gemini-embedding-001',
13     api_key=api
14 )
15
16 Settings.llm = llm
17 Settings.embed_model = embed_model
18 print('LLM e embeddings configurados')
```

... LLM e embeddings configurados

Código

2. Criando o índice vetorial, esse índice é criado sem o Chroma DB, diretamente em memória para simplificar a execução no Colab

[19]

✓ 1s

```
1 from llama_index.core import VectorStoreIndex
2 index = VectorStoreIndex(nodes, embed_model=embed_model)
3 print('Índice criado com sucesso')
```

▼

Índice criado com sucesso



[20]

✓ 0s



```
1 from llama_index.core.base.embeddings.base import similarity
2 #Realizando consultas no Chatbot
3 # query engine realiza consulta simples no documento
4 query_engine = index.as_query_engine(llm=llm,similarity_top_k=1)
5 response = query_engine.query("Quais grãos estão disponíveis")
6 print(response)
```

▼

... As variedades de café em grãos disponíveis são Bourbon vermelho, Bourbon amarelo, Blend especial (uma mistura de Bourbon amarelo e vermelho), Catuaí amarelo.

Código

[21]

✓ 0s

```
1 chat_engine = index.as_chat_engine(llm=llm,similarity_top_k=1)
2
```

[22]

✓ 1s



```
1 response = chat_engine.chat("Qual o valor de cada um")
2 print(response)
```



▼

... Olá! Na Serenatto Cafés Especiais, temos as seguintes variedades de café em grãos com seus respectivos preços:

```
* **Bourbon vermelho:** R$ 41,00
* **Bourbon amarelo:** R$ 43,00
* **Blend especial** (uma mistura de Bourbon amarelo e vermelho): R$ 37,50
* **Catuai amarelo:** R$ 55,00
* **Geisha:** R$ 105,00
* **Yirgacheffe:** R$ 110,00
```

Todos os nossos cafés são grãos torrados, provenientes de fazendas selecionadas em Minas Gerais, com nota SCA acima de 80, o que garante uma qualidade exce

Código

4. Modo Chat com memória resumida

```
1 from llama_index.core.memory import ChatSummaryMemoryBuffer
2 memory = ChatSummaryMemoryBuffer(llm=llm,token_limit=256)
3 chat_engine= index.as_chat_engine(
4     chat_mode = 'context',
5     llm = llm,
6     memory = memory,
7     system_prompt=(
8         'Você é especialista em cafés da loja Serenatto, uma loja online que vende'
9         'grãos de café torrados. Sua função é tirar dúvidas de forma simpática e natural sobre os grãos disponíveis'
10     )
11 )
```

```
1 response = chat_engine.chat('Olá')
2 print(response.response)
```

```
1 response = chat_engine.chat('Voce pode me dar detalhes sobre o Catui amarelo')
2 print(response.response)
```

Código

```
1 response = chat_engine.chat('Olá')
2 print(response.response)
```

```
1 response = chat_engine.chat('Voce pode me dar detalhes sobre o Catui amarelo')
2 print(response.response)
```

```
1 response = chat_engine.chat('qual o preço')
2 print(response.response)
```

```
1 response = chat_engine.chat('Voce pode me dar detalhes sobre o bourbon vermelho')
2 print(response.response)
```

```
1 response = chat_engine.chat('qual o preço citado antes')
2 print(response.response)
```

```
1 memory.get()
```

```
1 Comece a programar ou gere código com IA.
```

```
1 # Reset do chat
2 chat_engine.reset()
3 print('Chat resetado')
```

```
1 response = chat_engine.chat('O Neymar vai para a copa ')
2 print(response.response)
```

Código

```
1 # Criando a interface para o chatbot
2
3 ! pip install -q ipywidgets
```

```
1 import ipywidgets as widgets
2 from IPython.display import display, Markdown, clear_output
3 from google.genai.errors import ClientError
```

```
1 import ipywidgets as widgets
2 from IPython.display import display, Markdown, clear_output
3
4 # Se ClientError vier de alguma lib específica, mantenha o import correto.
5 # Exemplo:
6 # from google.api_core.exceptions import ClientError
7
8 # Área de saída do chat
9 chat_output = widgets.Output(
10     layout={
11         "border": "1px solid #ccc",
12         "padding": "10px",
13         "height": "400px",
14         "overflow_y": "auto"
15     }
16 )
17
18 # Área de entrada do chat
19 input_box = widgets.Text(
20     placeholder="Digite sua pergunta sobre a Serenatto",
21     description="Pergunta:",
```

Query Engine e Chat Engine

Característica	Query Engine	Chat Engine
Propósito	Recuperação de informações específicas	Interação conversacional contextualizada
Contexto	Não mantém contexto de conversa	Mantém contexto de conversa
Respostas	Diretas e baseadas em dados	Dinâmicas e contextualizadas
Uso Ideal	Perguntas pontuais e específicas	Conversas contínuas e interativas

Código

```
19 input_box = widgets.Text(  
20     placeholder="Digite sua pergunta sobre a Serenatto",  
21     description="Pergunta:",  
22     layout=widgets.Layout(width="80%")  
23 )  
24  
25 # Botões  
26 send_button = widgets.Button(  
27     description="Enviar",  
28     button_style="info" # troquei de primary para info  
29 )  
30  
31 clear_button = widgets.Button(  
32     description="Limpar",  
33     button_style="warning"  
34 )  
35  
36 # Histórico simples  
37 history = []  
38  
39 def responder(pergunta):  
40     try:  
41         response = chat_engine.chat(pergunta)  
42         return getattr(response, "response", str(response))  
43     except ClientError as e:  
44         erro = str(e)  
45  
46         if "429" in erro or "RESOURCE_EXHAUSTED" in erro:  
47             try:  
48                 retriever = index.as retriever(similarity top k=1)
```

Código

```
47     if "429" in erro or "RESOURCE_EXHAUSTED" in erro:
48         try:
49             retriever = index.as_retriever(similarity_top_k=1)
50             nodes = retriever.retrieve(pergunta)
51
52             if nodes:
53                 partes = []
54                 for i, node in enumerate(nodes, 1):
55                     texto = getattr(node, "text", str(node))
56                     partes.append(f"**Trecho {i}:**\n\n{texto[:500]}")
57
58                 return (
59                     "A API Gemini atingiu o limite no momento. "
60                     "Encontrei estes trechos locais:\n\n"
61                     + "\n\n---\n\n".join(partes)
62                 )
63
64             return "A API Gemini atingiu o limite e não encontrei trechos locais relevantes."
65
66         except Exception as erro_retriever:
67             return f"A API Gemini atingiu o limite e houve erro ao buscar trechos locais: {erro_retriever}"
68
69     return f"Erro na API: {erro}"
70
71 except Exception as e:
72     return f"Erro inesperado: {e}"
73
74 # Função para renderizar o chat
75 def render_chat():
76     with chat_output:
77         clear_output(wait=True)
```

Código

```
74 # Função para renderizar o chat
75 def render_chat():
76     with chat_output:
77         clear_output(wait=True)
78         display(Markdown("## Chatbot Serenatto"))
79
80         if not history:
81             display(Markdown("_Nenhuma mensagem ainda._"))
82
83         for role, content in history:
84             if role == "user":
85                 display(Markdown(f"**Você:** {content}"))
86             else:
87                 display(Markdown(f"**Serenatto:** {content}"))
88
89 # Evento de envio
90 def on_send(_):
91     pergunta = input_box.value.strip()
92     if not pergunta:
93         return
94
95     history.append(("user", pergunta))
96     resposta = responder(pergunta)
97     history.append(("assistant", resposta))
```

Código

```
99     input_box.value = ""
100     render_chat()
101
102 # Evento de limpar
103 def on_clear(_):
104     history.clear()
105     try:
106         chat_engine.reset()
107     except Exception:
108         pass
109     render_chat()
110
111 send_button.on_click(on_send)
112 clear_button.on_click(on_clear)
113 input_box.on_submit(lambda _: on_send(None))
114
115 display(widgets.VBox([
116     chat_output,
117     widgets.HBox([input_box, send_button, clear_button])
118 ]))
119
120 render_chat()
```


Código

Chatbot Serenatto

Você: oi

Serenatto: Olá! Como posso te ajudar hoje? Se tiver alguma dúvida sobre nossos cafés especiais, é só perguntar! 😊

Você: tipos de café

Serenatto: Na Serenatto, temos diversas variedades de cafés especiais, cada uma com suas características únicas! Você gostaria de saber sobre alguma em específico, ou prefere uma visão geral?

Temos opções como:

- **Bourbon Vermelho:** Doçura intensa, corpo encorpado e notas de chocolate.
- **Bourbon Amarelo:** Acidez vibrante com notas cítricas de maracujá, ameixa e nêspera.
- **Blend Especial:** Uma mistura equilibrada de Bourbon Vermelho e Amarelo, com notas de caramelo e chocolate.
- **Catuai Amarelo:** Perfil exótico com acidez expressiva e notas de acerola e pitanga.
- **Geisha:** Aroma floral com notas de jasmim e mamão papaya.
- **Yirgacheffe:** Acidez vibrante com notas de mel de laranja, chá de capim-limão e maçã fuji.

Me diga qual te interessa mais, ou se quer que eu detalhe algum deles! 😊

Pergunta:

Enviar

Limpar

Obrigado!

Prof. Me Daniel Vieira

Email: danielvieira2006@gmail.com

Linkedin: Daniel Vieira

Instagram: Prof daniel.vieira95

